Firmware Vulnerabilities Homology Detection Based on Clonal Selection Algorithm for IoT Devices

Daojing He, *Member, IEEE*, Xiaohu Yu, Tinghui Li, Sammy Chan, *Senior Member, IEEE*, and Mohsen Guizani, *Fellow, IEEE*

***Abstract*—With the wide application of Internet of Things (IoT) devices, security attacks against their firmware often occur, which has attracted more attention from the research community. Firmware is an important part of IoT devices, and attacks against them is one of the main means to destroy them. Therefore, firmware security is considered a core of the overall devices' security. At present, most of the firmware vulnerabilities have a small number of related samples, so it is difficult to use machine learning methods to generate detectors for some of them. Therefore, based on the collected data of related firmware vulnerabilities, this article proposes a firmware vulnerability homology detection method based on the clonal selection algorithm. We design the numerical and structural characteristics of vulnerability functions, train a detector for each function separately, and improve the recall rate of vulnerability detection. Compared with existing machine learning methods, this method only depends on the affinity between the objective function and the detector, which avoids the requirement of a large number of sample data sets. Finally, relevant experiments are carried out to verify the effectiveness of the method.**

***Index Terms*—Clonal selection, firmware, homology, Internet of Things (IoT).**

I. Introduction

IN RECENT years, with the rapid development of Internet of Things (IoT) people's daily life and industrial production are increasingly dependent on IoT equipment. According to a Gartner report [1], the number of IoT terminal devices has grown to 5.8 Billion in 2020, an increase of 21% over 2019. By the end of 2019, 4.8 Billion IoT terminals are expected to be put into use, an increase of 21.5% over 2018. The fact is that China alone had 4 Billion IoT devices in 2020. At the same time, there are increasing attacks on the firmware of IoT devices, and the current firmware vulnerabilities have not received enough attention. The American Consumer Institution (ACI) report [2] pointed out that five of every six (83%) wireless routers in the U.S. homes and offices have security vulnerabilities, and can lead to the risk of network attacks. After testing the wireless routers from 14 different equipment manufacturers, the researchers found that 155 (83%) of the 186 tested sample routers had different degrees of firmware security vulnerabilities, which might be the target of network attacks. At the same time, according to a Gartner report, about 70% of organizations that have not implemented firmware upgrade plans will be invaded due to firmware vulnerabilities by 2022.

Fuzzy testing [3]–[7], symbolic execution [8]–[13], taint analysis [14]–[18], and homology analysis [19]–[24] are four main means of firmware vulnerability analysis. At present, machine learning algorithms have achieved certain results in the field of firmware vulnerability detection [25], [26], but most of the research results are not ideal. Most of the experiments on firmware vulnerabilities are conducted on the firmware or open-source components crawled by researchers themselves. However, in the practical application environment, it is more valuable to correlate the sample data of firmware vulnerabilities, and the comparison of experimental results is more convincing. However, there is no relevant public data set at present. Therefore, we build a knowledge base related to firmware vulnerabilities, including vulnerability base, verification base, utilization base, patch base, and sample base. After completing the relevant vulnerability data collection, it is found that most of the firmware vulnerabilities only have a small number of relevant samples, which makes it difficult to use machine learning methods to generate detectors for some specific vulnerabilities. Therefore, based on the collected data of related firmware vulnerabilities, this article proposes a firmware vulnerability homology detection method based on the clonal selection algorithm, designs the numerical and structural characteristics of vulnerability functions, trains a detector for each function separately, and improves the

Manuscript received 20 August 2021; revised 23 November 2021 and 20 January 2022; accepted 7 February 2022. Date of publication 17 February 2022; date of current version 24 August 2022. This work was supported in part by the National Natural Science Foundation of China under Grant U1936120; in part by the National Key Research and Development Program of China under Grant SQ2021YFB2700900; in part by the Basic Research Program of State Grid Shanghai Municipal Electric Power Company under Grant 52094019007F; in part by the Fok Ying Tung Education Foundation of China under Grant 171058; and in part by the University Grants Committee of the Hong Kong Special Administrative Region, China, under Project CityU 11201421. *(Corresponding author: Daojing He.)*

Daojing He is with the Software Engineering Institute, East China Normal University, Shanghai 200062, China, and also with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, China (e-mail: hedaojinghit@163.com).

Xiaohu Yu and Tinghui Li are with the Software Engineering Institute, East China Normal University, Shanghai 200062, China (e-mail: 912200516@qq.com).

Sammy Chan is with the Department of Electrical Engineering, City University of Hong Kong, Hong Kong (e-mail: eeschan@cityu.edu.hk).

Mohsen Guizani is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, UAE (e-mail: mguizani@ieee.org).

Digital Object Identifier 10.1109/JIOT.2022.3152364

2327-4662 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.

recall rate of vulnerability detection. Compared with machine learning methods, this method only depends on the affinity between the objective function and the detector, which avoids the requirement of a large number of sample data sets.

Our contributions are summarized as follows.

1) Aiming at the analysis of large-scale firmware vulnerabilities, we study and analyze various techniques of firmware vulnerability analysis and compare the advantages and disadvantages. Then, we propose a new framework for homology detection, based on the clonal selection algorithm. In order to improve the local search ability, simulated annealing algorithm is introduced for further optimization.
2) We demonstrate the application of our method for vulnerability search in real-world applications. The experimental results show that the framework is feasible and has the highest recall rate compared with other methods.

The remainder of this article is organized as follows. We first introduce the research status of firmware security in Section II. In Section III, the homology detection of firmware vulnerabilities based on the clonal selection algorithm is proposed. In Section IV, experimental results show that the performance of the proposed method is better than previous methods. Finally, Section V summarizes this article.

II. Research Status and Problems of Firmware Security

Symbol execution has high requirements for hardware, and the corresponding symbol execution engine needs to be built for specific hardware. Therefore, it is not suitable for large-scale cross architecture firmware vulnerability analysis, and it is not suitable for large-scale automatic analysis due to the need to manually define policies and models. The path explosion of symbol execution also hinders its application. Similarly, fuzzy testing is also limited by hardware requirements, so it cannot achieve cross-architecture vulnerability analysis and requires manual participation. The current homology analysis technology has supported large-scale cross architecture firmware vulnerability identification, which can accurately and efficiently extract features and identify them.

Homology detection methods are mainly based on function (code block.), but it does not convey much information. These methods usually focus on functional control flow graph with attributes, semantics, attribute graph, and other objects for homology detection. Zhu *et al.* [22] proposed a foundation for cross-platform firmware binary code homology and similarity analysis by mining firmware code genes that can essentially identify code and exhibit stability, antivariability, and heritability, using the method of biological genes. Wang *et al.* [24] proposed IoTCMal, a hybrid IoT honeypot framework for capturing more comprehensive malicious samples aiming at IoT devices in 2019. Aiming at the problem that the method of homology detection based on file level feature is too rough, researchers start with the feature of function (code block) for homology detection. In order to solve the problems of too few firmware string features and difficult to extract firmware features using obfuscated encryption prevention measures, a binary file function feature coding method based on the neural network is proposed, starting from the function control flow graph with attributes. By encoding the static information features and structural features of the function contained in the binary file, the feature vector of the file is generated. At the same time, in order to improve the large-scale matching efficiency of feature database, the local sensitive hash method is used to improve the detection efficiency. The self-compiled test data set is tested, and the results show that it is 15%–20% higher than the previous work.

The semantics of function generalizes the logic of program execution and can be used in the field of homology detection. Lakhotia *et al.* [27] used the justice semantic extraction tool to quickly match code blocks according to their basic semantics. Pewny *et al.* [28] used semantic similarity principle to design the scheme. First, in order to enhance the robustness of different files, the semantics of binary files are represented by symbols. Then, the similarity of files is expressed by tree edit distance. On this basis, Pewny *et al.* [29] established signature for known vulnerabilities based on semantics, converted binary code into intermediate expression, and calculated similarity, which solved the problem of firmware vulnerability detection between different architectures to a certain extent. In view of the high time and space complexity of the semantic feature extraction process, Eschweiler *et al.* [30] started from known vulnerability functions to search for the same function from the target file, and generated function control flow graph based on it to calculate the similarity between them. At the same time, through the function feature preprocessing, the large-scale function level homology is realized, and the homology detection between different compilers, compilation parameters, operating systems, and architectures is realized.

Machine learning also has many applications in binary file detection [31]–[33]. Tommy *et al.* [34] described a system, which helps to automatically find and fix vulnerabilities in Web applications using machine learning Redmond *et al.* [35] introduced natural language processing into binary code analysis. In this method, they treated instructions as words in NLP-inspired binary code analysis, and aim to represent instructions as embeddings as well. They proposed a joint learning approach to generate instruction embeddings that capture not only the semantics. To solve the problem that most of the existing methods need to generate vectors manually, Ding *et al.* [36] proposed an assembly code representation learning model *Asm2Vec*. This model only needs to input assembly code and automatically identify the semantic relationship between the tags appearing in the merged assembly code. The experiments show that this model is more robust than existing methods. Additionally, aiming at the problem of manual feature extraction, Massarelli *et al.* [37] proposed a novel architecture, *SAFE*, to work directly on disassembled binary functions, not requiring manual feature extraction. Khedkar and AroulCanessane [38] presented machine learning algorithms for classifying the network traffic, for controlling the congestion in the network. Yosifova *et al.* [39] investigated the performance of base ML classifier algorithms, such as linear support vector classification, Naive Bayes, and random forest classifier. Their study showed that

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Clonal Selection Algorithm

```
Input: Random antibody population:Pr Size of the
       population:size Mature detector set:M Annealing
       temperature:Tt
Output: M
initialization:Tt=1 M;
while Tt>0 do
    P=Pr+M;
    Pn=Max(affinity);
    // Select n antibodies with the
       highest affinity value
    C=Clone(Pn);
    C*=Mutate(C);
    M=M+Selected(C*);
    // Select antibodies that meet the
       affinity threshold
    Tt=SA(Tt);// Simulated annealing
       algorithm
end
```

Algorithm 2: Simulated Annealing Algorithm

```
Input: Initial value:Y(i) Evaluation function value in
       state y:J(Y) Value that controls the rate of
       cooling:r
Output: Optimal solution:T
initialization;
while T>Tmin // If temperature T reaches
    Tmin, stop the search
do
    dE=J(Y(i+1))-J(Y(i));
    if dE<=0;
    // A better solution is obtained
       after moving
    then
        Y(i+1)=Y(i);
    else
        if exp(-dE/T)>random(0,1) then
            Y(i+1)=Y(i);
        end
    end
    T=r*T;
    // r represents the cooling rate
    i++;
end
```

machine learning is suitable for automated vulnerability type classification.

III. Homology Detection Based on Clonal Selection Algorithm

A. Firmware Vulnerabilities Homology Detection Model

The immune system cells can produce antibodies against invading antigens, and evaluate the antibodies by the specific binding degree of immunoglobulin between antibodies and antigens, so as to carry out the next step of immune cell reproduction and mutation. The cells with high tolerance to antigen were retained for replication, inheritance, and mutation, otherwise, the cells would die. After learning from this idea, researchers designed an immune clone algorithm. In this algorithm, the antigen and antibody are projected into the solution space, and the tolerance between the antigen and antibody is calculated by designing the affinity function of the solution space. The antibody with high affinity is retained and replaced by the antibody with low affinity. The process is repeated until the solution space approaches the objective function and the optimal solution is obtained. The goal of clonal selection algorithm is to cover the solution space of the objective function as much as possible.

This article designs a firmware vulnerability homology detection method based on clonal selection, combined with the simulated annealing algorithm. The pseudocodes of the clonal selection algorithm and simulated annealing algorithm are provided in Algorithms 1 and 2, respectively. Simulated annealing algorithm is a general optimization algorithm. It starts from a high initial temperature and randomly looks for the global optimal solution of the objective function in the solution space. The basic idea is to obtain the global optimal solution from the probability of a local optimal solution. Our method combines the excellent local search ability of simulated annealing algorithm and the fast convergence of clonal selection algorithm. Referring to Algorithm 1, the implementation steps of our method are as follows.

1) The initial antibody population P is generated randomly, and the size of the population is *size*. Each antibody in the population is encoded. The initial evolution algebra $t = 0$, the mature detector set M is initialized, and the initial annealing temperature $T_t = 1$.
2) The antigen affinity of antibody in immune cell population P is calculated according to the function samples.
3) According to the traditional clone immune algorithm, the antibody population is cloned, mutated, crossed, and selected.
4) The antibody population is divided into the quasimature antibody group P_r and nonmature antibody group M. Clone and inhibit the mature antibody group M, and update the immature antibody group P_r. The affinity was recalculated and the mature antibody was added to the mature detector set M.
5) We reach the annealing temperature, if $T_t \approx 0$, then the algorithm ends. Otherwise, continue to judge whether the detector set M meets the end condition. If so, complete the generation of detector and exit. Otherwise, go to step 2).

In the biological clonal selection algorithm, the immune system can recognize antigens and generate different plasma cells to produce antibodies according to the characteristics of different antigens. If the generated antibodies have high affinity with antigens, they will be retained, otherwise, they will be removed. In our case of IoT vulnerabilities, the antigen represents the optimization problem, the antibody represents the

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.

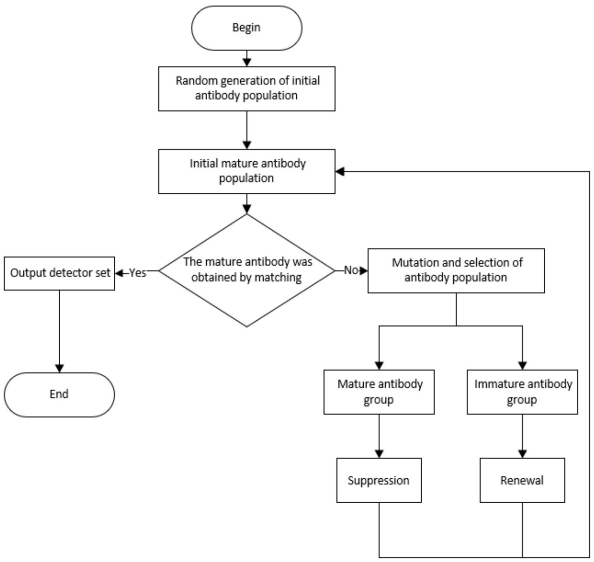

Fig. 1. Detector generation process of clonal selection algorithm.

feasible solution of the optimization problem, and affinity represents the quality of the feasible solution. In other words, the antigen is a vulnerability function, and the affinity represents the similarity between the sample function and the original function.

For the relevant vulnerability function samples, first, the function vulnerability detector is generated through the process shown in Fig. 1. A good detector will cover the solution space near the vulnerability function as much as possible. Therefore, in order to improve the coverage of the detector, this article sacrifices the efficiency of optimization, increases multiple rounds of clonal selection as much as possible, and selects new antibodies through annealing selection probability. After the detector set of vulnerability function samples is generated, the similarity is calculated with the samples extracted by the function to be detected, as shown in Fig. 2. Euclidean distance, Chebyshev distance, angle cosine distance, and Hamming distance are commonly used in similarity calculation. For example, Euclidean distance is used to calculate the distance between two points in N-dimensional Euclidean space. The normalized Euclidean distance is used to normalize the vector to counteract the influence of too large value of a certain dimension; Hamming distance is used to calculate the distance between strings or codes. In this article, the normalized Euclidean distance is used to calculate the similarity between the detection sample and the detector.

Simulated annealing algorithm is a random algorithm, which cannot necessarily find the global optimal solution, but can quickly find the near optimal solution of the problem. For the objective function of the simulated annealing algorithm, the time complexity is $O(n)$, and the time complexity of the whole algorithm depends on the outer loop of temperature control and the inner loop at a specific temperature, which is $O(abn)$. In this article, simulated annealing algorithm is used to find the optimal solution in clonal selection algorithm, so the time complexity of clonal selection algorithm is $O(n^2)$.

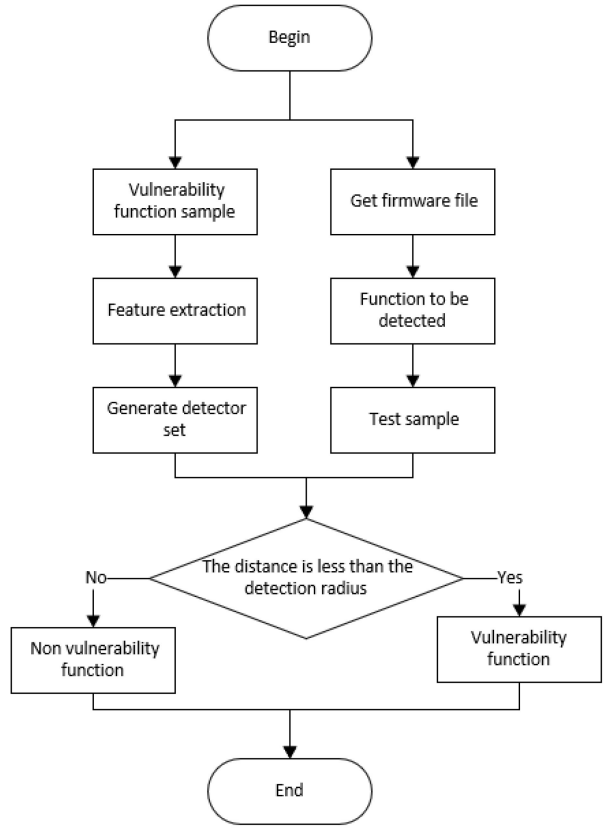

Fig. 2. Function matching process of clonal selection algorithm.

B. Feature Selection

In cross-platform homology detection, it is necessary to extract function features that are independent of platform and compiler, and reflect the internal relationship between functions. Therefore, after obtaining the firmware file, and unpacking and reversing the firmware, we can obtain the function characteristics of the binary file through the open-source tools *angr* and *IDAPython* plugin, including the numerical characteristics and structural characteristics of the firmware vulnerability function.

Binary files can generate a function control flow diagram and function call diagram, and remain unchanged in the cross-platform. Therefore, this article analyzes the function call graph, function control flow graph, and function basic information, and finally, obtains the following characteristics: the number of times a function calls other functions *callother*, and the number of times a function is called by other functions *becalled*. We use open-source tool *angr* to obtain function call graph and function control flow graph. Since many functions are called repeatedly, after reprocessing, the number of calls to other functions is *callothertwo*, and the number of calls is *becalledtwo*. We analyze the basic information of the function, and calculate the stack space used by the function, the number of strings, the number of instructions, the number of jump instructions, jump partition, and the proportion of jump instructions in the number of instructions. The function control flow graph is analyzed, and the number of basic blocks nodes and the number of edges between basic blocks

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.

are obtained. For the functional control flow graph, the minimum distance between the entry basic block and other basic blocks is calculated by the Dijkstra algorithm, and the longest path length of the graph is calculated by the classical depth first algorithm. When a directed graph is transformed into an undirected graph, the relationship between the variables they can express is different. Therefore, the function control flow graph is transformed into an undirected graph. By calculating the in degree and out degree of each node, the maximum values of in degree and out degree *inmax* and *outmax* and the average values of in degree and out degree *ineva* and *outeva* are calculated.

C. Operator

Based on the general clonal selection algorithm, we use a simulated annealing algorithm to find the optimal solution. In order to improve the coverage of the detector, we use multiple rounds of clonal selection, but due to the rapid convergence of simulated annealing algorithm, it has no impact on the efficiency. Compared with the common clonal selection algorithm, the method in this article has better local search ability and fast convergence.

The core of this method is the annealing selection probability. The calculation formula of annealing selection probability is as follows:

$$p(a_i) = \frac{e^{\frac{\text{affinity}(a_i)}{T_j}}}{\sum_i^n e^{\frac{\text{affinity}(a_i)}{T_j}}}.$$

The formula shows that the antibody a_i is selected with probability $p(a_i)$ and enters the next generation, where a_i represents the antibody, affinity(a_i) represents the affinity between the antibody and antigen, T_j represents the annealing temperature of generation j, j represents the current evolution algebra, n represents the maximum evolution algebra, and $T_j = T_{j-1} \times (1 - [j/n])$, the value of T_j will gradually tend to 0 during immunization.

The affinity function affinity(a_i) is used to calculate the affinity between antigen and antibody a_i. The function features used are all quantitative features, so the calculation formula of affinity function is as follows:

$$\text{affinity}(a_i) = \frac{1}{1 + H(t, a_i)}$$

where $H(t, a_i)$ represents the binding strength of antigen t and antibody a_i, and the calculation methods include Euclidean distance and Manhattan distance.

Annealing simulation algorithm is mainly composed of mutation operation and crossover operation. In the mutation operation, the cloned antibody is mutated and the related position in the antibody is XOR operated.

Other operations, such as selection operation, suppression operation, and update operation, operate the antibody according to the traditional clonal selection algorithm. Wherein, the antibody concentration is expressed by formula $C(a_i) = ([\delta(a_i)]/\text{size})$. Among them, $\delta(a_i)$ represents the number of antibodies in the field δ of antibody a_i, and *size* indicates the number of antibody populations.

IV. Experimental Analysis of Clonal Selection Algorithm

A. Data of Experiment

At present, researchers use the self-crawling firmware or self-compiled third-party library to detect the vulnerability homology, and there is no systematic arrangement of firmware vulnerability-related resources. Compared with the research on how to improve the accuracy of vulnerability detection and retrieval speed, the application value of firmware vulnerability correlation technology depends more on the basic data sets related to firmware vulnerabilities. In the later stage, we can increase the verification, utilization, and other functions, provide repair suggestions, and improve the information level of the system. At the same time, because there is no public data set for researchers to use, the comparison of data results among researchers is affected by certain programs, so this article first collects and processes the relevant vulnerability data set, and carries out open-source processing for other researchers to use.

In this article, the relevant data of firmware vulnerability are sorted out, and the knowledge base of firmware vulnerability is established. Standards related to vulnerability include CVE (public vulnerability and exposure), CVSS (general vulnerability scoring system), and CPE (general platform enumeration). Among them, CVE is a description list for known security vulnerabilities, including identification code, description, and public reference; CVSS is an open framework to evaluate vulnerability severity. It quantifies vulnerability characteristics from the dimensions of foundation, time, and environment, and calculates the value of vulnerability severity. The latest version is cvssv3.1. This article uses CVSS V2.0, CVSS V3.0 version standard to calculate the vulnerability evaluation score. CPE is a standardized description method for describing and identifying applications, operating systems and hardware in digital assets. The version adopted in this article is CPE2.3.

By 2020, the number of CVE vulnerabilities has reached 131 906, among which 1689 are related to firmware, and the trend is increasing year by year. The vulnerability inventory stores the vulnerability information of CVE and cnvd official platforms from 1999 to 2020. At present, 139 145 pieces of data are collected, and it supports automatic crawling of vulnerability entry information to keep the current vulnerability database in the latest state.

For some vulnerabilities related to system information, vulnerability detection and vulnerability verification need to match the corresponding system information and software information, and system information extraction is a necessary step. For IoT devices running online, scanning tools, such as nmap and OpenVAS scanner, can be used to scan the target devices and their network segments. It can also get the information of device operating system, file system, application service version, and configuration file through remote login and offline detection. Open vulnerability and evaluation language (OVAL) completely defines the system information related to vulnerability. Through the standardized vulnerability verification and test definition of OVAL, the possible vulnerability of the target system can be verified. We have built a vulnerability verification library based on OVAL. In the later

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.

simulation environment, we can use the OVAL vulnerability verification library to detect and verify vulnerabilities.

A database of exploit assemblies provided by "official security" is built in order to establish exploit library. Exploit programs are stored in different directories according to their corresponding manufacturers (ASUS, Cisco, Hikvision, Dahua, etc.). The exploit DB vulnerability library contains four types of vulnerability Codes: webapps, remote, DOS, local, and shellcode. Among them, Webapps are Web application vulnerability exploiters, such as SQL injection, XSS, etc.; remote is a remote exploitable program, such as remote buffer overflow, remote command execution, etc.; DOS is a program that exploits DOS type vulnerabilities, such as memory corruption, stack overflow, etc.; local is the exploiter of local vulnerability, such as local privilege raising and privilege promotion; and shellcode is shellcode of different platforms.

The patch strategy library is a library of strategies, methods, and tools for repairing the discovered firmware vulnerabilities. The patch strategy library contains the specific repair methods and resource files of the vulnerability patches. The patch policy library will be connected with the vulnerability library in real time, and the patch and repair method will be updated at the same time. In the patch policy library, there are not only patches and policies but also patch files of vulnerabilities, which are numbered and stored in the form of files. The patches of vulnerabilities can be found through the attributes of vulnerabilities.

A vulnerability sample set is the core of firmware vulnerability homology detection, so according to the vulnerability information of firmware, through crawling, manual information collection, and processing, more than 300 basic data sets related to firmware vulnerabilities have been collected from technical papers, official websites and other sources, including source code, configuration files, binary files, code fragments, etc. The fields in the database style sheet include CVE-id, filetype, files, MD5, etc., which are finally stored in the unstructured database mongodb.

B. Result of Experimental Analysis

We selected 400 samples of firmware test under MIPS, ARM, and other architectures from the D-Link router firmware crawled from the network. The firmware is unpacked by *binwalk*, *factextractor*, *binarynalysisng*, and other open-source tools, and the related binary files are extracted. The binary file is retrieved, and the binary files that may have homology with the vulnerability to be detected are obtained by searching the Web, cgibin, cgi, and other fields. Finally, a total of 368 binary files are obtained, including 52 624 functions, among which the number of vulnerability functions with the same name as the D-Link router firmware vulnerability is 30, 42, 28, and 29, respectively. By manual verification, the number of functional vulnerabilities with problems is determined to be 28, 30, 27, and 22. We take the binary file of corresponding firmware version as the sample to generate detector of the vulnerability function.

We compare the clonal selection algorithm with the neural network algorithm. In order to measure the difference between neural network algorithm and the clonal selection algorithm, recall rate and accuracy are used as evaluating indicator. Recall rate is the ratio between the number of real vulnerabilities in the vulnerability set determined by the algorithm and the number of real vulnerabilities to be detected. Accuracy is the ratio between the number of real vulnerabilities in the vulnerability set determined by the algorithm and the number of vulnerabilities predicted by the algorithm.

TABLE I
COMPARISON OF EXPERIMENTAL RESULTS

Name of Function	method of neural network		our method	
	recall rate	accuracy	recall rate	accuracy
alpha_auth_check	0.62	0.75	0.82	0.76
get_input_entries	0.68	0.71	0.77	0.73
authenticationcgi_main 0.80	0.75	0.78	0.82	0.83
hedwigcgi_main	0.73	0.77	0.84	0.76

Due to the need to generate detectors for some specific vulnerabilities, the vulnerability detection using the neural network method needs a long running time. However, the method in this article only depends on the affinity between the objective function and the detector, hence, it avoids the requirements of a large number of sample data sets, and can significantly reduce the operation time.

The final experimental results are shown in Table I. The recall rate of our method is significantly better than that of the compared method, which is improved by about 13%, and can find more loophole functions. The accuracy is slightly improved, but the effect is not obvious.

V. SUMMARY

In order to improve the application value of a firmware vulnerability homology detection, this article has established a knowledge base related to firmware vulnerability, including verification base, utilization base, patch base, and sample base, and carries out open-source processing of relevant data. At the same time, in view of the fact that there are too few data sets of relevant vulnerability functions in the actual sample library, the neural network algorithm cannot effectively train vulnerability functions to establish detectors, and a clonal selection method based on the simulated annealing algorithm is designed to establish detectors for each vulnerability function. The experimental results show that the method is effective in detecting the homology of firmware vulnerabilities.

However, the amount of data in vulnerability verification library and sample library is too small, so more data collection and manual extraction can be done at a later stage. Although the clonal selection algorithm improves the recall rate, it does not significantly improve the accuracy rate. In order to improve the accuracy, we can try to apply deep learning in the future.

The core idea of simulated annealing algorithm originated from statistical thermodynamics and is suitable for the model and theoretical description of statistical physics. The work of this article is not deep enough in describing the time complexity of the proposed algorithm. In future work, we plan to further explore its mathematical and physical models, in order to describe in detail the time and space complexity of the simulated annealing algorithm, and further optimize

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.

it. In addition, vulnerability detection technologies based on homology analysis will be based on syntax and semantics in the future. These methods will still use machine learning to match features, including control flow diagram and data flow diagram. Firmware samples are difficult to obtain, which makes it difficult for machine learning to train large-scale data sets. Therefore, other algorithmic assisted machine learning methods will become the mainstream research direction of homology analysis technology, which is also our research direction in the future.

REFERENCES

[1] Gartner, Stamford, CT, USA. "IoT Adoption Trends: Where Your Competitors Are Investing," Apr. 2021. [Online]. Available: https://www.gartner.com/en/innovation-strategy/trends/iot-adoption-trends

[2] ACI, Scottsdale, AZ, USA. "Securing IoT Devices: How Safe Is Your Wi-Fi Router?" Apr. 2021. [Online]. Available: https://www.theamericanconsumer.org/wp-content/uploads/2018/09/FINAL-Wi-Fi-Router-Vulnerabilities.pdf

[3] C. Chen, B. Cui, J. Ma, R. Wu, J. Guo, and W. Liu, "A systematic review of fuzzing techniques," *Comput. Secur.*, vol. 75, pp. 118–137, Jun. 2018.

[4] S. Paruchuri, A. Case, and G. Richard, III, "Gaslight revisited: Efficient and powerful fuzzing of digital forensics tools," *Comput. Secur.*, vol. 97, Oct. 2020, Art. no. 101986.

[5] D. Cotroneo, A. K. Iannillo, and R. Natella, "Evolutionary fuzzing of android OS vendor system services," *Int. J. Empir. Softw. Eng.*, vol. 24, pp. 3630–3658, May 2019.

[6] T. Nguyen-Anh and Q. Le-Trung, "RFL-IoT: An IoT reconfiguration framework applied fuzzy logic for context management," in *Proc. IEEE-RIVF Int. Conf. Comput. Commun. Technol. (RIVF)*, Danang, Vietnam, 2019, pp. 1–6.

[7] Z. Gui, H. Shu, and J. Yang, "FIRMNANO: Toward IoT firmware fuzzing through augmented virtual execution," in *Proc. IEEE 11th Int. Conf. Softw. Eng. Service Sci. (ICSESS)*, Beijing, China, 2020, pp. 290–294.

[8] T. Zhang, Y. Zhang, Z. Chen, Z. Shuai, and J. Wang, "Efficient multiplex symbolic execution with adaptive search strategy," in *Proc. 35th IEEE/ACM Int. Conf. Automat. Softw. Eng. (ASE)*, Melbourne, VIC, Australia, 2020, pp. 1255–1256.

[9] S. Shen, S. Ramesh, S. Shinde, A. Roychoudhury, and P. Saxena, "Neuro-symbolic execution: The feasibility of an inductive approach to symbolic execution," Jul. 2018, *arXiv:1807.00575*.

[10] J. Wu, C. Zhang, and G. Pu, "Reinforcement learning guided symbolic execution," in *Proc. 27th IEEE Int. Conf. Softw. Anal. Evol. Reeng.*, London, U.K., 2020, pp. 662–663.

[11] S. Germiniani, A. Danese, and G. Pravadelli, "Automatic generation of assertions for detection of firmware vulnerabilities through alignment of symbolic sequences," *IEEE Trans. Emerg. Topics Comput.*, early access, Nov. 2, 2020, doi: 10.1109/TETC.2020.3035187.

[12] F. Fowze, D. J. Tian, G. Hernandez, K. Butler, and T. Yavuz, "ProXray: Protocol model learning and guided firmware analysis," *IEEE Trans. Softw. Eng.*, vol. 47, no. 9, pp. 1907–1928, Sep. 2021.

[13] N. Corteggiani and A. Francillon, "HardSnap: Leveraging hardware snapshotting for embedded systems security testing," in *Proc. 50th Annu. IEEE/IFIP Int. Conf. Depend. Syst. Netw. (DSN)*, Valencia, Spain, 2020, pp. 294–305.

[14] A. M. Alashjaee, S. Duraibi, and J. Song, "IoT-Taint: IoT malware detection framework using dynamic taint analysis," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, Las Vegas, NV, USA, 2019, pp. 1220–1223.

[15] D. E. Denning, "A lattice model of secure information flow," *Commun. ACM*, vol. 19, pp. 236–243, May 1976.

[16] M. Yi, X. Xu, and L. Xu, "An intelligent communication warning vulnerability detection algorithm based on IoT technology," *IEEE Access*, vol. 7, pp. 164803–164814, Nov. 2019.

[17] N. Redini *et al.*, "Karonte: Detecting insecure multi-binary interactions in embedded firmware," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Francisco, CA, USA, 2020, pp. 1544–1561.

[18] M. Li, Y. Lu, H. Huang, C. Zhang, and J. Zhao, "Research on automatic exploitation of house of spirit heap overflow vulnerability" in *Proc. IEEE 4th Inf. Technol. Netw. Electron. Autom. Control Conf. (ITNEC)*, Chongqing, China, 2020, pp. 751–757.

[19] J. Bauwens, P. Ruckebusch, S. Giannoulis, I. Moerman, and E. De Poorter, "Over-the-air software up-dates in the Internet of Things: An overview of key principles," *IEEE Commun. Mag.*, vol. 58, no. 2, pp. 35–41, Feb. 2020.

[20] L. Ran *et al.*, "An experimental study of four methods for homology analysis of firmware vulnerability," in *Proc. Int. Conf. Depend. Syst. Appl. (DSA)*, Beijing, China, 2017, pp. 42–50.

[21] J. Liu, Y. Shen, and H. Yan, "Functions-based CFG embedding for malware homology analysis," in *Proc. 26th Int. Conf. Telecommun. (ICT)*, 2019, pp. 220–226.

[22] X. Zhu, Q. Li, P. Zhang, and Z. Chen, "A firmware code gene extraction technology for IoT terminal," *IEEE Access*, vol. 7, pp. 179591–179604, Dec. 2019.

[23] X. Zhu, Q. Li, Z. Chen, G. Zhang, and P. Shan, "Research on security detection technology for Internet of Things terminal based on firmware code genes," *IEEE Access*, vol. 8, pp. 150226–150241, Aug. 2020.

[24] B. Wang, Y. Dou, Y. Sang, Y. Zhang, and J. Huang, "IoTCMal: Towards A Hybrid IoT Honeypot for Capturing and Analyzing Malware," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Dublin, Ireland, 2020, pp. 1–7.

[25] Y. Liu, J. Wang, J. Li, S. Niu, and H. Song, "Machine learning for the detection and identification of Internet of Things (IoT) devices: A survey," Jan. 2021, *arXiv:2101.10181*.

[26] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: A survey," *Proc. IEEE*, vol. 108, no. 10, pp. 1825–1848, Oct. 2020.

[27] A. Lakhotia, M. Preda, and R. Giacobazzi, "Fast location of similar code fragments using semantic juice," in *Proc. 2nd ACM SIGPLAN Program Protect. Reverse Eng. Workshop ACM*, New York, NY, USA, 2013, pp. 1–6.

[28] J. Pewny, F. Schuster, L. Bernhard, T. Holz, and C. Rossow, "Leveraging semantic signatures for bug search in binary programs," in *Proc. 30th Annu. Comput. Secur. Appl. Conf.*, New Orleans, LA, USA, 2014, pp. 406–415.

[29] J. Pewny, B. Garmany, R. Gawlik, C. Rossow, and T. Holz, "Cross-architecture bug search in binary executables," in *Proc. IEEE Symp. Secur. Privacy*, San Jose, CA, USA, 2015, pp. 709–724.

[30] S. Eschweiler, K. Yakdan, and E. Gerhards-Padilla, "DiscovRE: Efficient cross-architecture identification of bugs in binary code," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, 2016. [Online]. Available: http://dx.doi.org/10.14722/ndss.2016.23185

[31] S. Laha, N. Chowdhury, and R. Karmakar, "How can machine learning impact on wireless network and IoT?–A survey," in *Proc. 11th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, 2020, pp. 1–7.

[32] K. Sharma and R. Nandal, "A literature study on machine learning fusion with IOT," in *Proc. 3rd Int. Conf. Trends Electron. Informat. (ICOEI)*, 2019, pp. 1440–1445.

[33] Z. Jin and Y. Yu, "Current and future research of machine learning based vulnerability detection," in *Proc. 8th Int. Conf. Instrum. Meas. Comput. Commun. Control (IMCCC)*, 2018, pp. 1562–1566.

[34] R. Tommy, G. Sundeep, and H. Jose, "Automatic detection and correction of vulnerabilities using machine learning," in *Proc. Int. Conf. Current Trends Comput. Elect. Electron. Commun. (CTCEEC)*, 2017, pp. 1062–1065.

[35] K. Redmond, L. Luo, and Q. Zeng, "A cross-architecture instruction embedding model for natural language processing-inspired binary code analysis," Dec. 2018, *arXiv:1812.09652*.

[36] S. H. H. Ding, B. C. M. Fung, and P. Charland, "Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2019, pp. 472–489.

[37] L. Massarelli, G. A. Di Luna, F. Petroni, R. Baldoni, and L. Querzoni, "SAFE: Self-attentive function embeddings for binary similarity," in *Proc. Detect. Intrusions Malware Vulnerabil. Assessment*, 2019, pp. 309–329.

[38] S. P. Khedkar and R. AroulCanessane, "Machine learning model for classification of IoT network traffic," in *Proc. 4th Int. Conf. I-SMAC (IoT Social Mobile Anal Cloud) (I-SMAC)*, 2020, pp. 166–170.

[39] V. Yosifova, A. Tasheva, and R. Trifonov, "Predicting vulnerability type in common vulnerabilities and exposures (CVE) database with machine learning classifiers," in *Proc. 12th Nat. Conf. Int. Participat. (ELECTRONICA)*, 2021, pp. 1–6.

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.

Daojing He (Member, IEEE) received the B.Eng. and M.Eng. degrees in computer science from Harbin Institute of Technology, Harbin, China, in 2007 and 2009, respectively, and the Ph.D. degree in computer science from Zhejiang University, Hangzhou, China, in 2012.

He is currently a Professor with the Software Engineering Institute, East China Normal University, Shanghai, China, and also with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. His research interests include network and systems security.

Prof. He is on the editorial board of some international journals, such as *IEEE Communications Magazine* and IEEE NETWORK.

Xiaohu Yu was born in 1998. He is currently pursuing the master's degree with Software Engineering Institute, East China Normal University, Shanghai, China.

Tinghui Li was born in 1995. He is currently pursuing the master's degree with Software Engineering Institute, East China Normal University, Shanghai, China.

Sammy Chan (Senior Member, IEEE) received the B.E. and M.Eng.Sc. degrees in electrical engineering from the University of Melbourne, Melbourne VIC, Australia, in 1988 and 1990, respectively, and the Ph.D. degree in communication engineering from the Royal Melbourne Institute of Technology, Melbourne, in 1995.

Since December 1994, he has been with the Department of Electrical Engineering, City University of Hong Kong, Hong Kong, where he is currently an Associate Professor.

Mohsen Guizani (Fellow, IEEE) received the B.S. (with distinction) and M.S. degrees in electrical engineering, and the M.S. and Ph.D. degrees in computer engineering from Syracuse University, Syracuse, NY, USA, in 1984, 1986, 1987, and 1990, respectively.

He is currently with the Mohamed Bin Zayed University of Artificial Intelligence, Abu Dhabi, UAE.

Dr. Guizani is a Senior Member of ACM.

Authorized licensed use limited to: Tsinghua University. Downloaded on November 12,2022 at 17:40:45 UTC from IEEE Xplore. Restrictions apply.